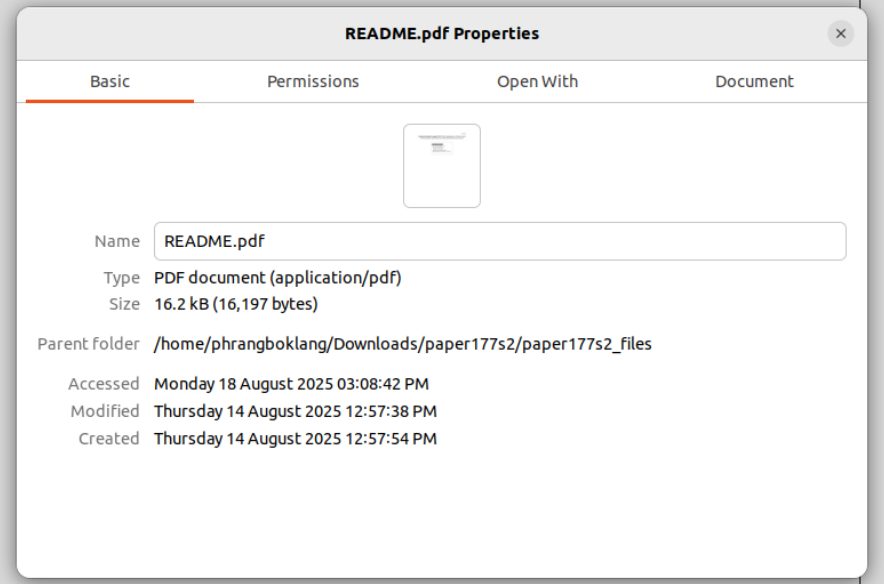

FILES AND DIRECTORIES

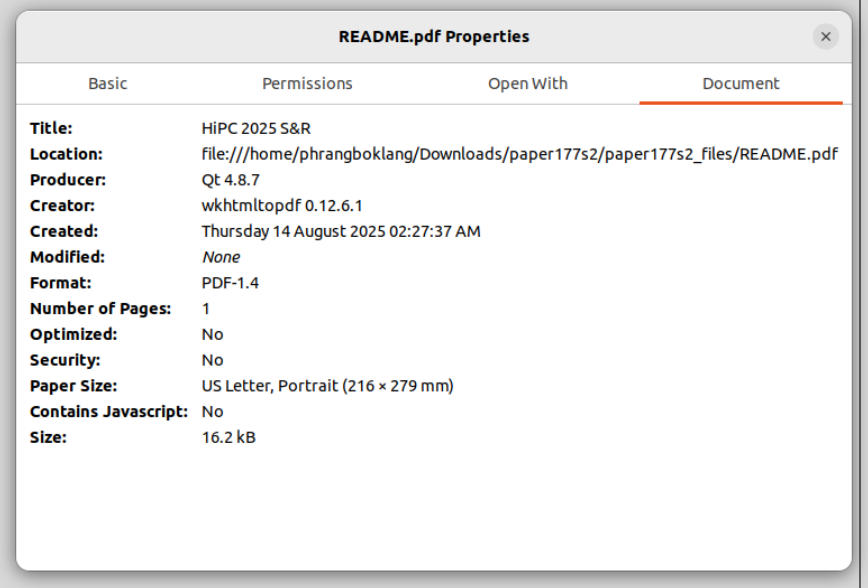
File Concept

- Contiguous Logical Address Space
  - Files are viewed as a linear sequence of bytes, forming a continuous logical address space.
- File Types (Defined by Creator):
- Data Files
  - *Numeric*: e.g., spreadsheets, sensor logs
  - *Character*: e.g., text documents
  - *Binary*: raw or formatted binary data
- Program Files
  - Executables, scripts, and source code
  - Examples of File Types:
    `->.txt`, `.cpp`, `.exe`, `.bin`

File Attributes

- Files have associated metadata used by the file system for management and access:
  - **Name** – Human-readable name to identify the file
  - **Identifier** – Unique ID within the file system
  - **Type** – Defines file format and associated applications
  - **Location** – Pointer to file's physical location on storage
  - **Size** – File's total storage requirement (in bytes)
  - **Protection** – Access permissions (read/write/execute)
  - **Timestamps** – Creation/modification/access times, user ID
- File attributes are stored in the **directory structure**, which resides on disk.





File Types and Common Extensions

File Type	Typical Extensions	Description
Source Code	.c, .cpp, .java, .py	Program source code in high-level languages
Object	.obj, .o	Compiled machine code (not yet linked)
Executable	.exe, .out, .bin	Ready-to-run binary program
Text	.txt, .md	Plain text documents
Batch	.bat, .sh	Scripts for command execution
Word Processor	.docx, .odt, .rtf	Formatted text documents
Spreadsheet	.xls, .xlsx, .ods	Tabular data files
Database	.db, .sql, .mdb	Structured data storage
Image	.jpg, .png, .gif, .bmp	Digital images and graphics
Multimedia	.mp3, .mp4, .avi, .mkv	Audio and video files
Archive	.zip, .rar, .tar, .gz	Compressed collections of files
PDF	.pdf	Portable Document Format

File Operations

- Create: Allocate space and add file entry to the directory.
- Write: Add data at the current write pointer location; update pointer accordingly.
- Read: Retrieve data from the current read pointer location; advance pointer.
- Seek: Move the read/write pointer to a specific location within the file.
- Delete: Remove file entry from directory and free its disk space.
- Truncate: Erase file contents but retain the file name and metadata.
- Open(F): Locate file F in the directory, load metadata into memory (open file table).
- Close(F): Write updated file metadata from memory back to the disk directory structure.

Managing Open Files

- Several key data structures are used to track open files:
  - Open-File Table
    Maintained by the OS; stores metadata for all currently open files.
  - File Pointer
    Per-process pointer to the current read/write location within the file.
  - File-Open Count
    Tracks how many processes have the file open. Used to determine when the file can be removed from the open-file table.
  - Disk Location
    Cached information about where the file resides on disk, speeding up access.
  - Access Rights
    Per-process permissions (e.g., read, write, execute) enforced by the OS.

File Locking Mechanisms

- Locks mediate concurrent access to files by multiple processes:
- **Shared Lock** *(Reader Lock)*
  - Multiple processes can hold the lock simultaneously for reading.
- **Exclusive Lock** *(Writer Lock)*
  - Only one process can hold the lock; used for writing.
- **Locking Modes:**
  - **Mandatory Locking**
    - Enforced by the OS: access is denied if the lock is incompatible.
  - **Advisory Locking**
    - Optional: processes check the lock status and cooperate voluntarily.

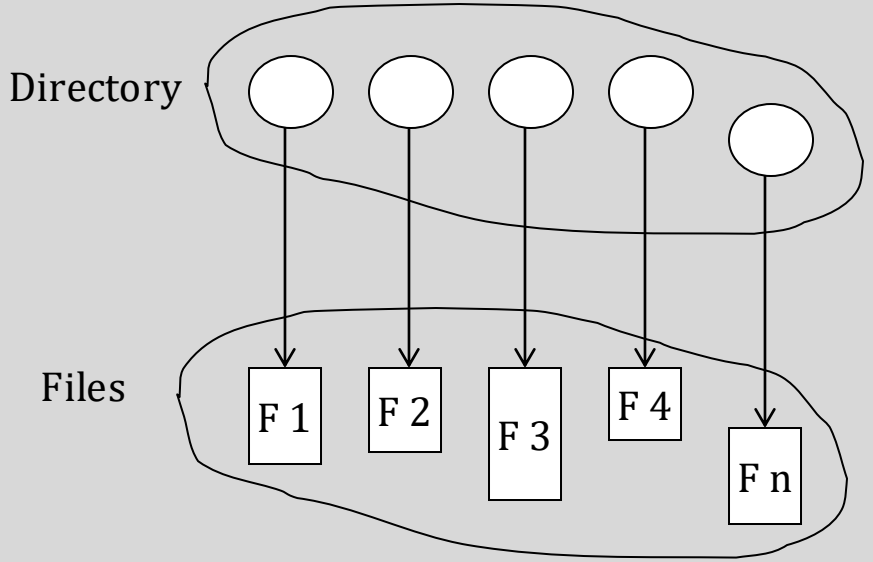
File Access Methods

- **Sequential Access**
  - Data is accessed one record after another in order.
  - Simple and efficient for streaming or batch processing.
  - Common for text files, logs, and backups.

- **Direct (Random) Access**
  - Allows immediate access to any record by calculating its address.
  - Enables jumping directly to data locations.
  - Used in databases, index files, and multimedia applications.
- Sequential access is linear and simple, while direct access is fast and flexible.

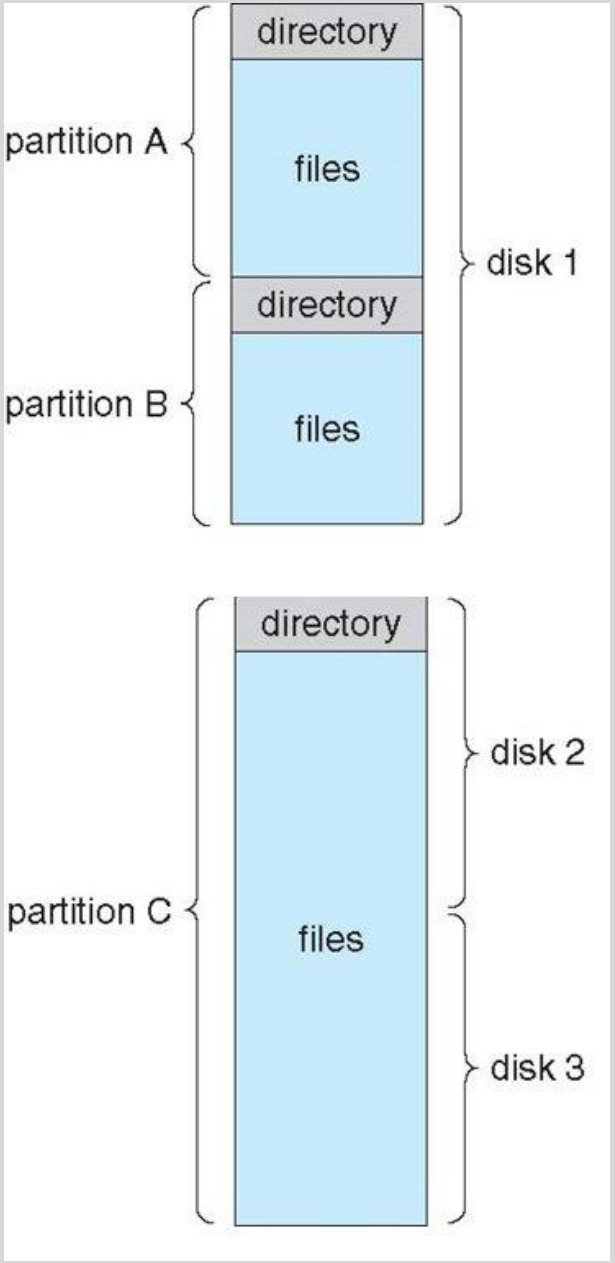
Directory Structure

- A collection of nodes that store metadata about all files.
- Organizes file information like names, locations, sizes, and attributes.
- Both the directory structure and the actual files are stored on disk.
- Acts as a map for the file system to locate and manage files efficiently.



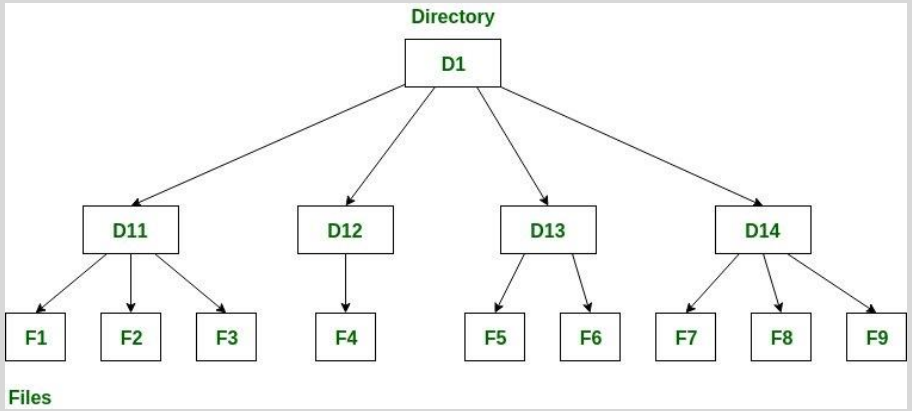
Disk Structure

- A **disk** can be divided into multiple **partitions** (also called minidisks or slices).
- Each partition can be **RAID protected** to improve fault tolerance and reliability. (RAID to be discussed later).
- Partitions may be used **raw** (without a file system) or **formatted** with a file system.
- Each partition holds a **file system volume** that manages files and metadata.
- The volume's metadata is stored in a **device directory** or **volume table of contents** for tracking.



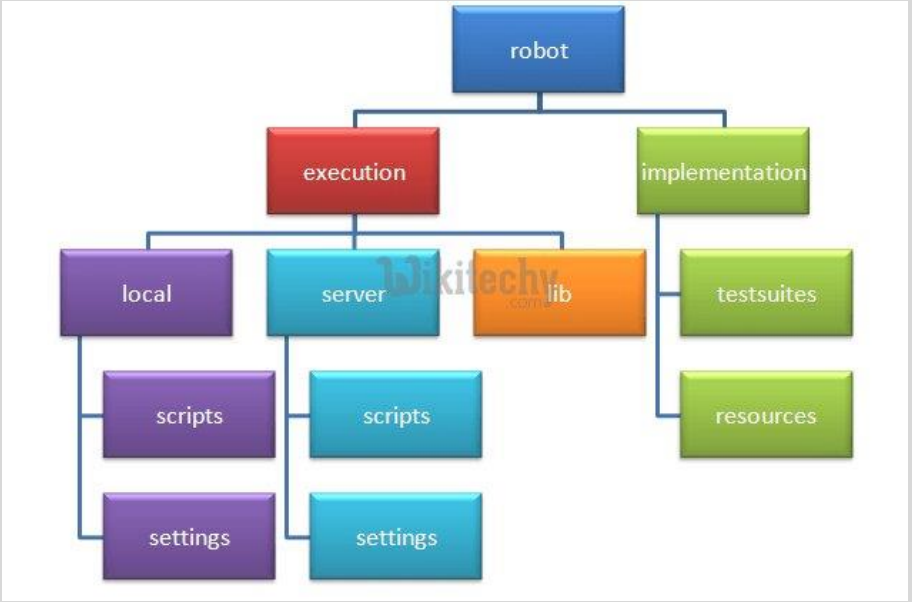
Operations Performed on a Disk

- Search for a File
  Locate a file by name or metadata within the directory structure.
- Create a File
  Allocate space and add a new file entry to the directory.
- Delete a File
  Remove file entry and free its storage space on disk.
- List a Directory
  Display contents (files and subdirectories) within a directory.
- Rename a File
  Change the name associated with a file's directory entry.
- Traverse the File System
  Navigate through directories and subdirectories to access files.



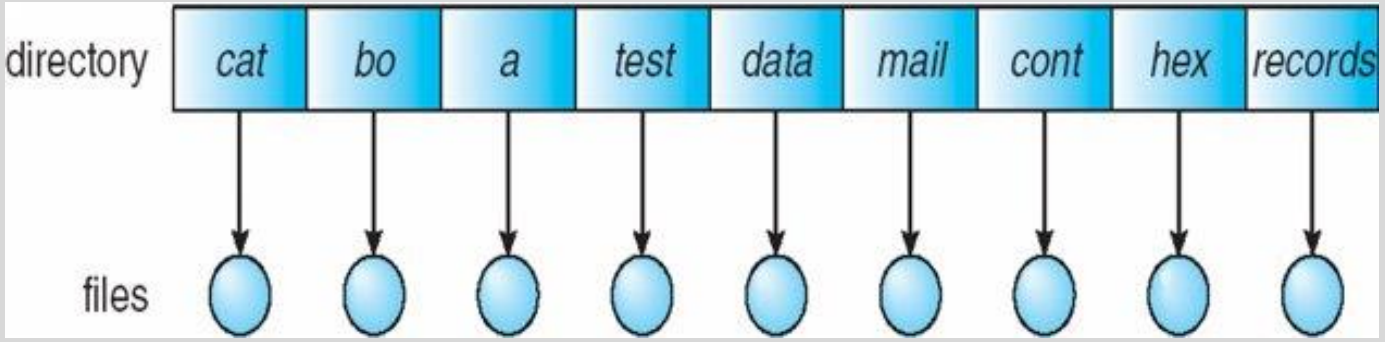
Directory Structure Concepts

- **Efficiency**
  Quickly locate files within the system.
- **Naming**
  Provide convenient, human-friendly file names.
- **Name Flexibility**
  - Different users can have files with the same name.
  - A single file can have multiple names (hard links).
- **Grouping**
  Logical organization of files by type or purpose (e.g., all programs, all games).



Single-Level Directory

- **Structure:**
  One common directory contains all files for all users.
- **Problems:**
- **Naming Conflict:** Different users cannot have files with the same name.
- **Poor Grouping:** No way to logically group files by user or type, leading to clutter.

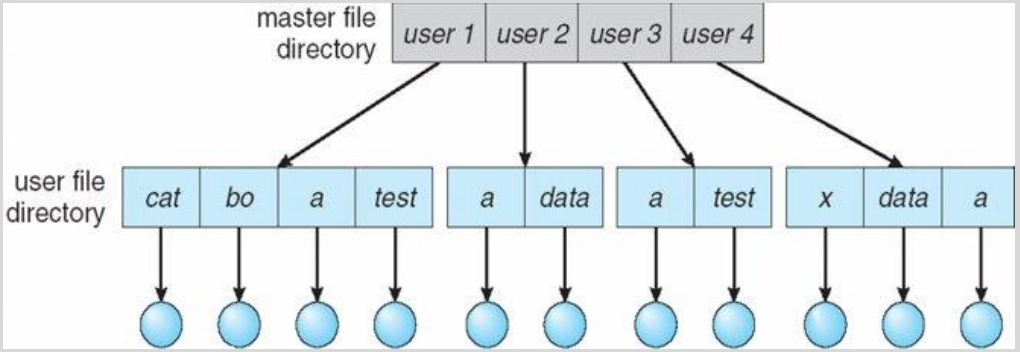


Two-Level Directory

- Each user has a **separate directory** for their files.
- Files are accessed using a **path name**:

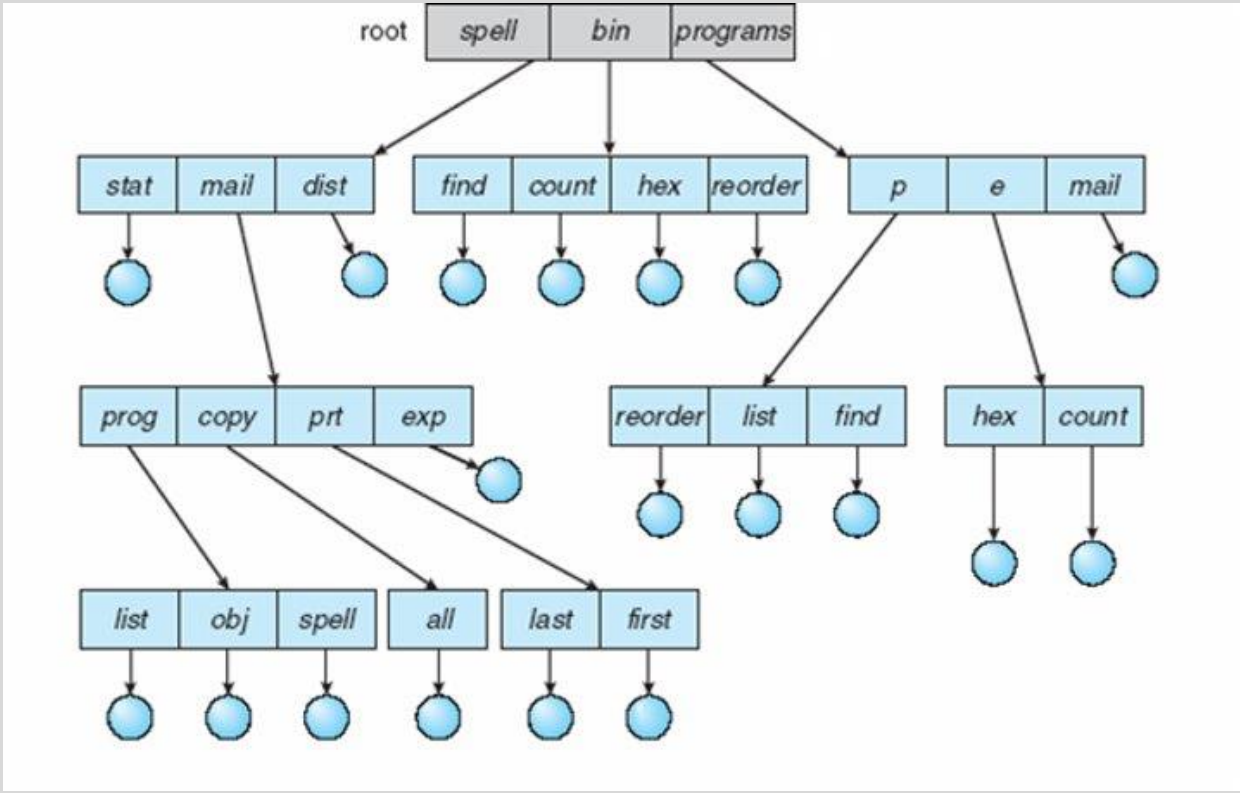
  `UserName/FileName`

- Allows **same file names** for different users without conflict.



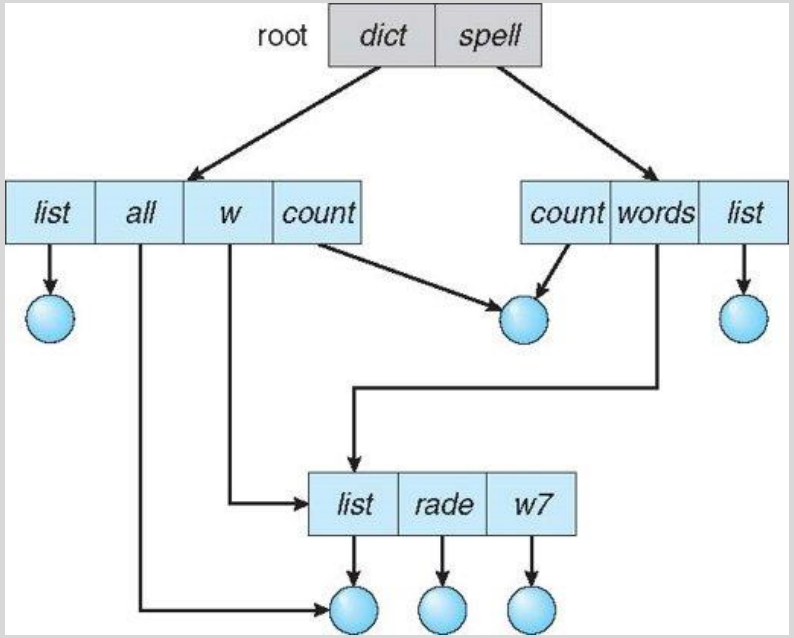
Tree-Structured Directories

- Files are organized in a **hierarchical tree** of directories and subdirectories.
- Each directory can contain files and other directories (subdirectories).
- Allows **flexible and logical grouping** of files.
- Path names specify the location of files within the hierarchy (e.g., `/home/user/docs/file.txt`).
-



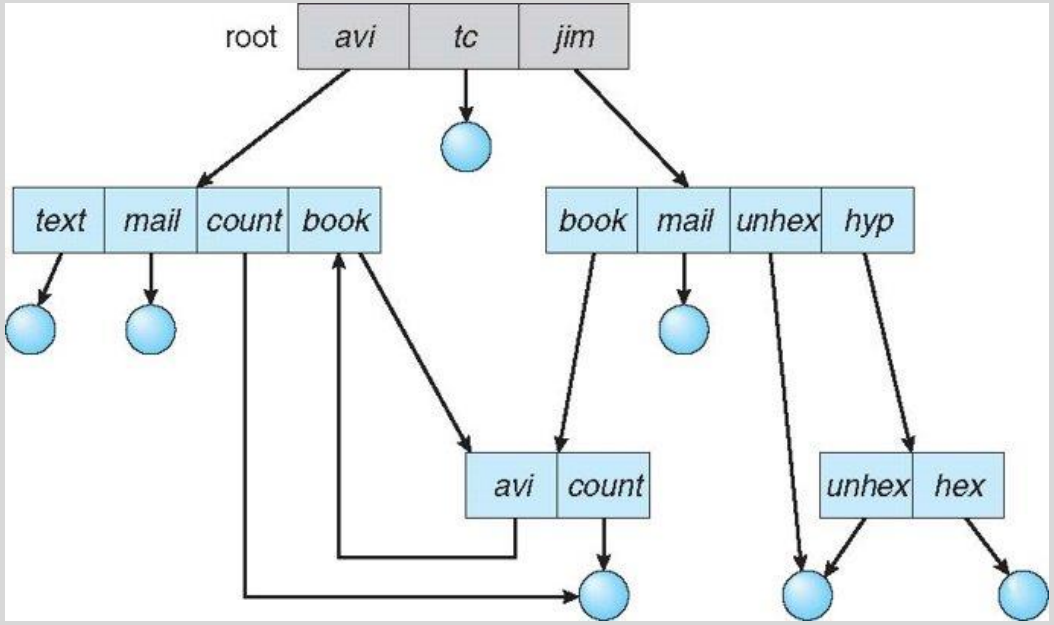
Acyclic-Graph Directories

- Directory structure forms a **directed acyclic graph (DAG)**, not just a tree.
- **Allows sharing of files and subdirectories** by multiple parent directories.
- **No cycles** allowed to prevent infinite loops in traversal.
- Supports **hard links** to files and directories (with restrictions).
- Enables better file sharing and efficient storage.



General Graph Directory

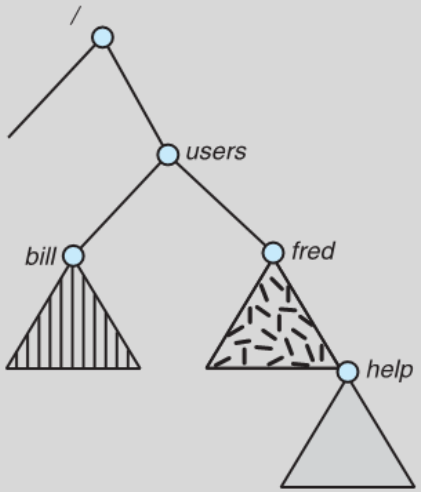
- Directory structure forms a **general graph**, allowing **cycles**.
- Enables **full flexibility**: files and directories can be linked arbitrarily.
- **Cycles can cause issues** like infinite loops during traversal.
- Requires **special handling** to detect and prevent problems (e.g., reference counting, garbage collection).
- Supports **symbolic (soft) links** that can create cycles.



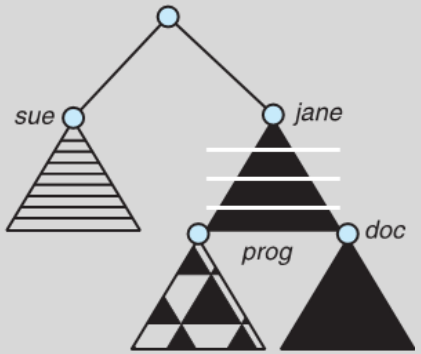
File System Mounting

- **Mounting** is the process where the OS makes files and directories on a storage device accessible through the file system.
- The OS gains access to the device, reads, and processes its file system structure and metadata.
- The location in the file system where the device is attached is called the **mount point**.
- After mounting, users can access the device's files and directories starting from the mount point.
- The OS **recognizes the storage medium** and integrates it into the file system hierarchy.
- Mounting enables **transparent access** to different file systems through a unified directory tree.

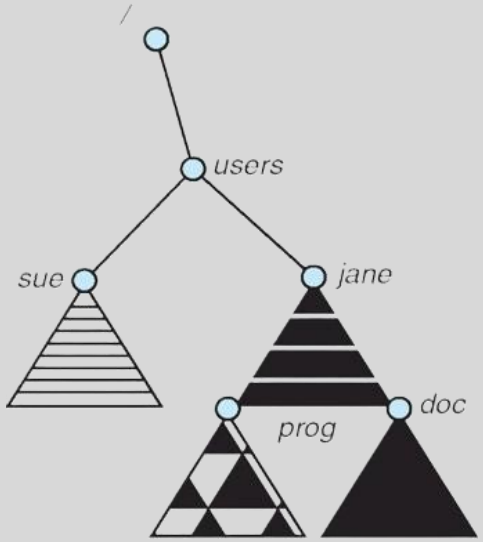
Example


/
users
bill
fred
help

(a)


sue
jane
prog
doc

(b)


/
users
sue
jane
prog
doc

File Sharing in Multi-User Systems

- **File sharing** enables multiple users to access files, which is essential in multi-user environments.
- Sharing is controlled through **protection schemes** to enforce access permissions.
- In **distributed systems**, files can be shared across networks using protocols like **Network File System (NFS)**.
- **User IDs (UIDs)** uniquely identify users, enabling per-user access control.
- **Group IDs (GIDs)** allow users to be organized into groups, facilitating group-based permissions.
- Each file or directory has an **owner** and an associated **group**, which determine access rights.

File Sharing – Remote File Systems

- Enables **file system access across a network** between different systems.
- Can be done in several ways:
  - **Manual access** via tools like **FTP** (File Transfer Protocol).
  - **Automatic & seamless** access through **distributed file systems** (e.g., NFS).
  - **Semi-automatic** access via the **World Wide Web** (e.g., downloading files via HTTP).
- Based on the **client-server model**:
  - Clients **mount remote file systems** exported by servers.
  - Remote files appear as part of the local directory structure.

Remote File Access & Naming Services

- **Standard file operations** (open, read, write) are **translated into remote procedure calls** over the network.
- **NFS (Network File System):**
  - Standard client-server file sharing protocol on **UNIX/Linux** systems.
- **CIFS (Common Internet File System):**
  - File sharing protocol used by **Windows** systems (also known as SMB).
- **Distributed Information Systems** (naming services):
  - Provide **unified access** to user, system, and file info across a network.
  - Examples:
    - **LDAP** (Lightweight Directory Access Protocol)
    - **DNS** (Domain Name System)
    - **NIS** (Network Information Service)
    - **Active Directory** (Windows environments)

Example: Modern File Sharing & Distributed File Systems

- **Amazon EFS (Elastic File System)**
  - Fully managed, scalable **cloud-based NFS** file system on AWS.
  - Used in cloud-native applications.
- **Google Filestore / Azure Files**
  - Managed file storage services on **GCP** and **Azure**, respectively.
  - Offer SMB/NFS-based remote access with cloud integration.

Failure Modes in File Systems

- All file systems are prone to **failures** (e.g., power loss, disk crash, metadata corruption).
- **Remote file systems** introduce additional failure modes:
  - **Network failure** (connectivity loss, latency)
  - **Server failure** (remote server crashes or restarts)
- **Failure recovery** requires:
  - Tracking the **state of each remote request** (e.g., whether it completed, needs retry)
- **Stateless protocols** (like NFSv3):
  - Include all necessary info in **each request** (no server-side session tracking)
  - Easier to recover after failure (retransmit request)
  - **Less secure** and may cause performance overhead

Protection

- The **file owner/creator** controls:
  - **Who** can access the file
  - **What actions** are permitted
- Common **types of access**:
  - **Read:** View contents
  - **Write:** Modify contents
  - **Execute:** Run as a program
  - **Append:** Add data at the end
  - **Delete:** Remove the file
  - **List:** View directory contents (for directories)

Access Modes & User Classes (Unix/Linux)

- **Modes of Access:**
  - **Read (r):** View file contents
  - **Write (w):** Modify file contents
  - **Execute (x):** Run file as a program
- **Three User Classes:**
  - **Owner:** Creator of the file
  - **Group:** Users belonging to a specific group
  - **Public (Others):** Everyone else
- **Group Management:**
  - A manager creates a group (G) and adds users to it.
  - Files can have permissions assigned separately for owner, group, and others.

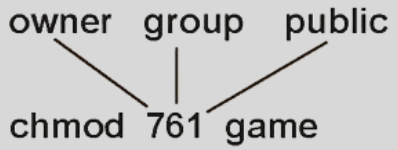
User Class	Permissions	Meaning
Owner	rwx	Full control
Group	r-x	Read and execute only
Others	r--	Read only

RWX

a) **owner access** $7 \Rightarrow 1\ 1\ 1$

b) **group access** $6 \Rightarrow 1\ 1\ 0$

c) **public access** $1 \Rightarrow 0\ 0\ 1$

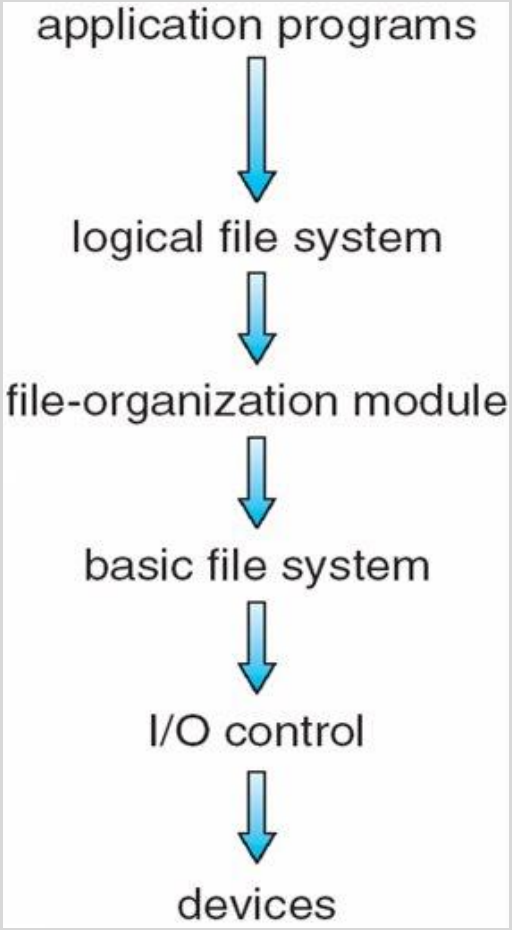


Overview of File Systems

- A **file** is the logical unit of storage.
- The **file system** resides on secondary storage (disks).
- It provides a user interface that **maps logical file operations to physical disk storage**.
- Enables **efficient and convenient access** by allowing easy storage, location, and retrieval of data.
- The **disk** provides the physical space for files.
- I/O transfers occur in **blocks of sectors**, typically 512 bytes each.
- A **File Control Block (FCB)** stores metadata and management information about each file.

Layered File System

- **Logical File System:**
  - Manages **metadata** about files.
  - Translates **file names** into file numbers, file handles, and locations using **File Control Blocks (FCBs)**.
  - Handles **directory management** and **protection**.
- **File Organization Module:**
  - Understands the file's structure, logical addresses, and physical blocks.
  - Translates **logical block numbers** to **physical block numbers** on disk.

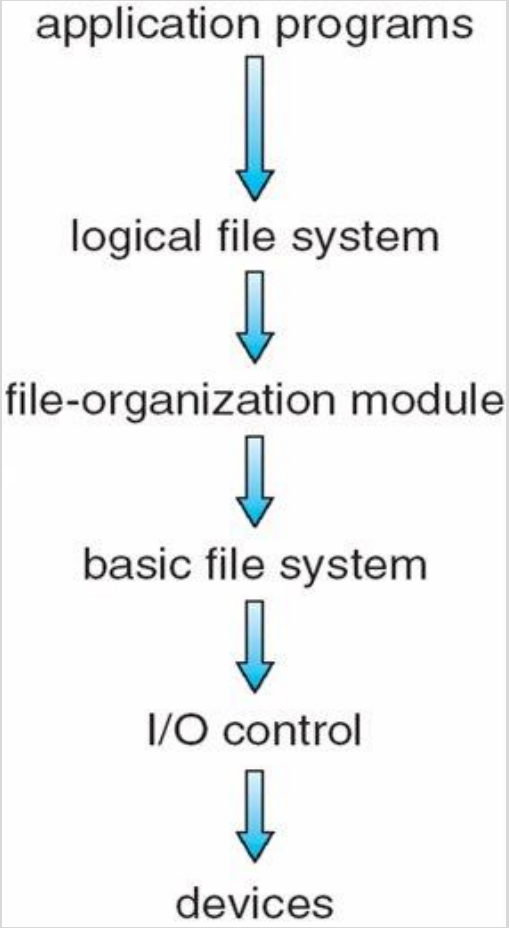


Layered File System (continued)

- **Basic File System:**
  - Issues generic I/O commands to device drivers.
  - Example:

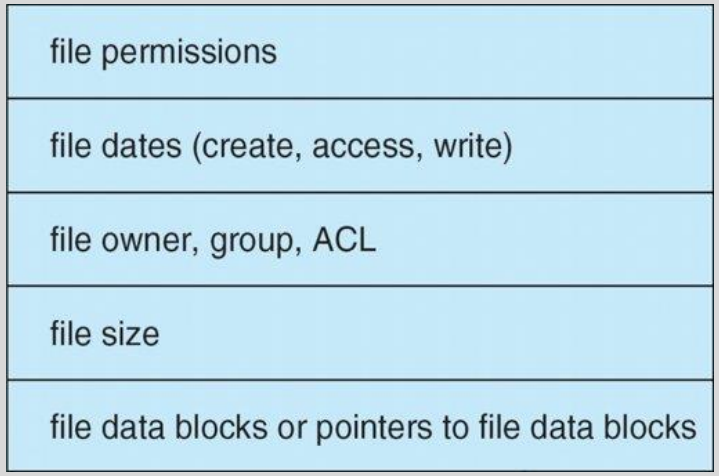
```
read drive2, cylinder 45, track 3, sector 7, into memory
location 2048
```

- **Device Drivers:**
  - Manage I/O devices at the **I/O control layer**.
  - Translate generic commands into **hardware-specific, low-level instructions** for the device controller.



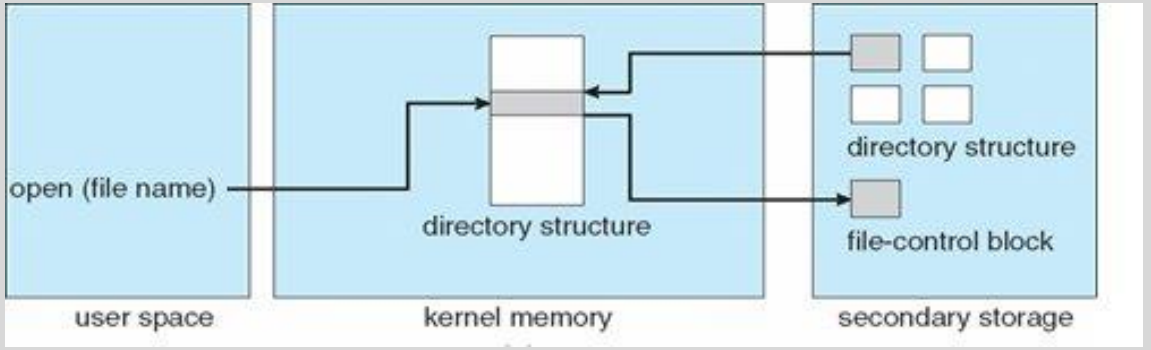
File System Implementation: File Control Block (FCB)

- The **File Control Block (FCB)** stores essential details about a file, including:
  - **Inode number** (unique file identifier)
  - **Permissions** (read, write, execute rights)
  - **File size**
  - **Timestamps** (creation, modification, access dates)
- FCB is key for managing files efficiently within the file system.



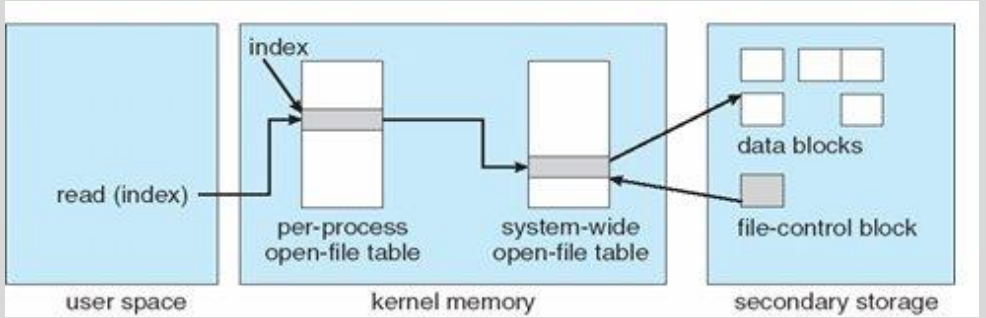
In-Memory File System Structures

- When a process **opens** a file, the operating system creates an **in-memory file handle** (also called a file descriptor).
  - This handle uniquely identifies the open file instance for that process.
  - It is used for subsequent operations like reading, writing, or closing the file.
- The OS maintains an **open-file table** in memory:
  - Tracks all open files system-wide.
  - Stores information like the file pointer (current read/write position), access mode, and reference counts.



In-Memory File System Structures (Continue)

- During a **read** operation:
  - Data is first retrieved from disk into the file system's **buffer cache** (memory).
  - Then, data is copied from the buffer cache into the **user process's memory** at the address specified in the read call.
- These in-memory structures improve performance by:
  - Avoiding repeated disk access (using caching)
  - Keeping track of file status efficiently across processes.



reading a file

File-System Implementation

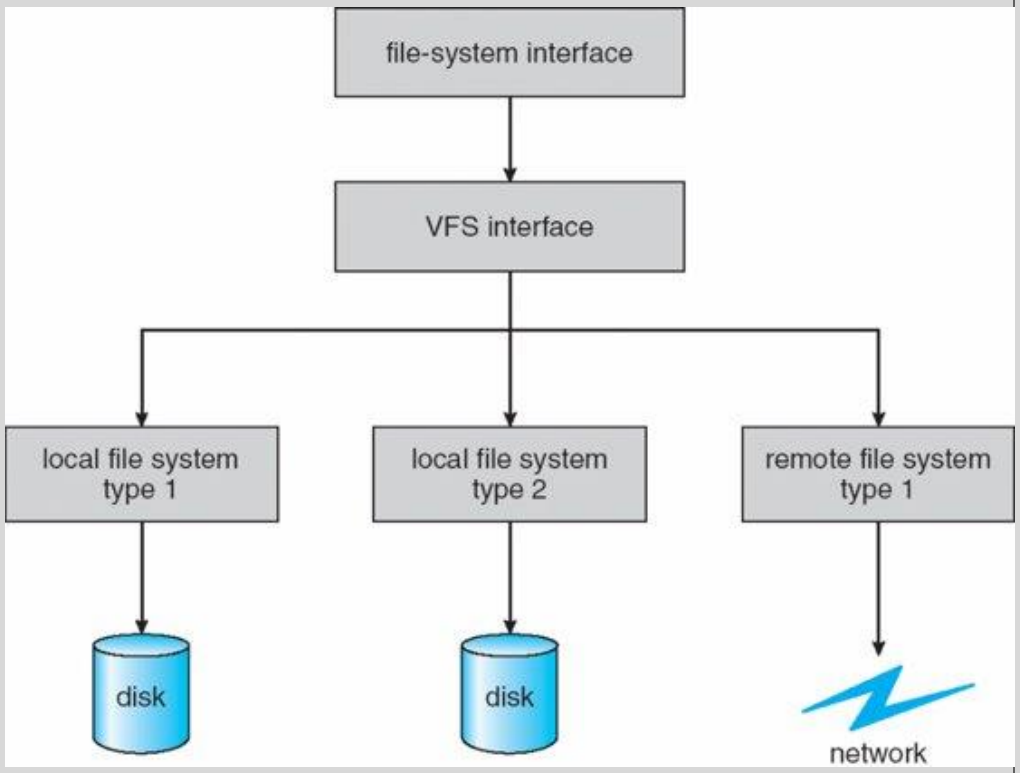
- **Boot Control Block:**
  - Contains information needed to **boot the OS** from that volume.
  - Usually located in the **first block** of the volume.
- **Volume Control Block (Superblock / Master File Table):**
  - Stores key volume details:
    - Total number of blocks
    - Number of free blocks
    - Block size
    - Pointers or arrays tracking free blocks
- **Directory Structure:**
  - Organizes files by associating **file names with inode numbers**.
  - Master File Table maintains this mapping and metadata.

Partitions and Mounting

- A **partition** can be
  - A volume with a **file system**
  - Or just a sequence of blocks without a file system (raw)
- The **boot block**
  - Points to the **boot volume** or contains a **boot loader**, code that loads the OS kernel from the file system
  - May also be a **boot manager** for multi-OS booting
- The **root partition** holds the OS; other partitions may contain
  - Additional OSes
  - Different file systems
  - Raw data (no file system)
- Partitions are **mounted** at boot time or manually later
  - During mounting, **file system consistency is checked**
  - If metadata is corrupted, the system tries to **fix it** and retries
  - If consistent, the partition is added to the **mount table** and becomes accessible

Virtual File Systems

- VFS is an **abstraction layer** that provides a uniform interface to different file systems.
- Applications use the **VFS API** for file operations like open, read, write, regardless of the underlying file system type.
- Enables support for **multiple file systems simultaneously** (local, network, virtual).
- Simplifies file system development by separating user interface from file system specifics.
- VFS manages generic structures (inodes, directory entries, superblocks) to represent files and directories.
- Translates generic file requests into specific calls for the appropriate file system driver.
- Allows transparent access to files across different mounted file systems through a single interface.



VFS Object Types & Operations (Linux Example)

- Linux VFS defines four main object types:
  - **inode:** Represents file metadata
  - **file:** Represents an open file instance
  - **superblock:** Represents a mounted file system
  - **dentry:** Directory entry, links names to inodes
- VFS defines a standard set of operations that must be implemented by each file system:
  - `int open(...)` — Open a file
  - `int close(...)` — Close an open file
  - `ssize_t read(...)` — Read from a file
  - `ssize_t write(...)` — Write to a file
  - `int mmap(...)` — Memory-map a file

Directory Implementation

- **Linear List of File Names:**
  - Each entry contains a file name and a pointer to the data blocks.
  - Simple to implement but inefficient for large directories due to **linear search** time.
- **Ordered List:**
  - Entries kept alphabetically using linked lists or **B+ trees** to speed up search.
- **Hash Table:**
  - Uses a hash function to map file names to locations in the directory.
  - Reduces average search time significantly.
  - Must handle **collisions** (e.g., via chaining or overflow areas).
  - Works best with fixed-size entries or by using overflow handling.

Thank You